

CEHv13 A.I MODULE 11- Session Hijacking

Notes by VIVEK GOSWAMI

Overview:

Session hijacking is an attack where an attacker exploits a valid computer session to gain unauthorised access to information or services in a system.

It can be categorised as either active or passive based on the attacker's level of involvement:

1. Active Session Hijacking:

- The attacker identifies an active session between a client and a server.
- The attacker takes over the session by injecting malicious packets or commands.
- Objective: To impersonate the legitimate user and gain control of the session.

2. Passive Session Hijacking:

- The attacker monitors and records traffic between the victim and the server without interfering.
- No direct control over the session is taken.
- Objective: To capture sensitive data such as login credentials, cookies, or session tokens.

What is Session Hijacking?

Session hijacking is an attack that exploits valid computer sessions to gain unauthorised access. The attacker steals or predicts a valid session ID/token to gain unauthorised access to an authenticated user's session with a server.

What is Session ID (Token)?

A unique identifier assigned by a web server to a client after successful authentication. This token is used to maintain the session state and differentiate multiple user sessions.

After validating a user based on their username and password credentials, the server assigns a unique string value, known as a Session ID, to maintain the authenticated state.

Key Characteristics:

User Identification: Session IDs are used to identify authenticated users

Fresh Assignment: A new Session ID is assigned after each successful authentication.

Storage Method: Session IDs are typically stored in browser cookies

Temporary Nature: Valid only for the duration of the user session

COOKIES IN SESSION MANAGEMENT

When a user logs in, the server creates a session and sends a unique session ID back to the user's browser as a cookie. This cookie is then sent back to the server with subsequent requests, allowing the server to recognise the user and maintain their authenticated state throughout the session.

Key points about cookies in session management:

- Cookies store session tokens that uniquely identify a user session on the server.
- Session cookies are typically non-persistent and expire when the browser is closed, enhancing security.
- Cookies can hold user-specific data like authentication status and preferences for continuous session tracking.
- Regular session ID rotation and session expiration reduce the risks of session hijacking.
- Improperly configured cookies (e.g., lacking security flags) can expose sessions to attacks like XSS and CSRF, leading to unauthorised access or session compromise.

COOKIES V/S SESSION V/S TOKEN

1. Cookies:

- Small pieces of data stored on the client (browser).
- Typically used to store session identifiers or tokens.
- Automatically sent with every HTTP request to the server.
- Can be configured with security flags (HttpOnly, Secure, SameSite).
- Serve as a transport mechanism for session IDs or tokens.

2. Sessions:

- Server-side storage of user-specific data, including authentication state.
- Associated with a unique session ID stored on the client (usually in a cookie).
- The session ID is a random, hard-to-guess value that links client requests to server-side data.
- Sessions maintain user state across multiple requests securely by keeping data server-side.
- Session expiration and invalidation are handled by the server.

3. Tokens:

- Self-contained pieces of data (often JWTs) that carry authentication and authorization info.
- Stored on client-side (can be in local storage, session storage, or cookies).
- Usually stateless; the server may not store session data for tokens.
- Contain encoded and sometimes signed user info to verify authenticity.
- Used commonly in API or mobile app authentication; sent typically in HTTP headers (e.g., Authorisation).

→ Cookies are the storage and transport method on the client.

→ Sessions are server-side state management using session IDs transported via cookies.

→ Tokens are client-stored credentials carrying user claims, useful for stateless authentication.

How Does Session Hijacking Work?

- **The Set-Up:** You log into a website and send a session token to your computer. Think of it as a secret handshake that helps the website identify you.
- **The Steal:** While browsing, a hacker intercepts the communication and steals your session token. This can happen if the hacker is on the same network or if the website doesn't protect the token well.

- **The Takeover:** With your session token, the hacker can pretend to be you. They can access your account, see your private information, and even act like you.

Why Session Hijacking Works

Common Vulnerabilities

1. Insecure Session Handling
 - Weak session ID generation algorithms
 - Predictable session tokens
 - Insufficient randomness in token creation
2. Insecure Session Termination
 - Sessions are not properly invalidated on logout
 - Long session timeouts
 - No automatic session cleanup
3. Weak Session ID Generation
 - Linear algorithms using time or IP addresses
 - Sequential numbering systems
 - Insufficient entropy in random generation
4. Unencrypted Session Transmission
 - Session IDs transmitted over unencrypted connections
 - Missing HTTPS enforcement
 - Vulnerable to network interception

Types of Session Hijacking

1. Active Session Hijacking

Active Session Hijacking involves the attacker directly stealing session cookies from the victim's browser and using those cookies to impersonate the user.

Characteristics:

- Direct Cookie Theft: Extraction of session cookies via malware or XSS
- Application-Level: Operates at the application layer
- Immediate Usage: Stolen sessions used immediately for unauthorized access
- Higher Detection Risk: May leave traces in application logs

Passive Session Hijacking

Passive Session Hijacking uses network sniffers and monitoring tools to obtain session information, allowing attackers to log in as valid users.

Characteristics:

Network-Level: Operates at the network layer

Stealth Operation: Difficult to detect as no direct interaction

Traffic Analysis: Monitors network traffic for session tokens

Delayed Usage: Sessions may be used later to avoid detection

Hands-on Labs for Session Hijacking

Lab 1: Capturing Session ID via Wireshark

- Monitor network traffic using Wireshark.
- Apply filter: `http` to capture HTTP requests.
- Identify session cookies by inspecting HTTP request headers for "Cookie" fields.

Lab 2: Hijacking a Session with Cookie Injection

- Use Burp Suite to intercept HTTP traffic and extract session cookies.
- Open browser developer tools and locate the "Application" tab (Chrome) or equivalent.
- Modify the session cookie value in the browser to the extracted cookie.
- Refresh or revisit the target web application to access the victim's session.

Lab 3: Sidejacking over HTTP

- Capture unencrypted session tokens via packet capture tools (Wireshark, tcpdump).
- Extract captured tokens from HTTP headers.
- Replay the captured token by injecting it into your browser's session cookie to gain unauthorized access to the target account.

Session Hijacking Attack Methods

Session Prediction

Analyzing session ID patterns to predict valid session tokens for other users.

Man-in-the-Middle (MITM) Attacks

Intercepting network traffic between client and server to capture session tokens.

Man-in-the-Browser (MITB) Attacks

Using malware like BeEF (Browser Exploitation Framework) to hijack victim's browser session.

Network Sniffing

Monitoring network traffic to capture unencrypted session tokens.

Malware Attacks

Using trojans and spyware to steal session cookies directly from victim's system.

Cross-Site Scripting (XSS)

Executing malicious scripts to extract session cookies when victims browse compromised websites.

Proxy Server Attacks

Using attacker-controlled proxy servers to intercept and analyze all victim traffic.

Tools for Session Hijacking

1. **Wireshark:** A Packet capture and network traffic analysis tool used for sniffing session tokens and cookies.
2. **Ettercap:** A man-in-the-middle (MITM) attack tool that supports network sniffing and poisoning attacks.
3. **Burp Suite:** Web proxy and interception tool useful for capturing and manipulating web traffic, including session cookies.
4. **Cain & Abel:** Network sniffer and password recovery tool capable of capturing credentials and session information.
5. **Bettercap:** Advanced MITM tool that supports comprehensive network attacks and session hijacking capabilities.
6. **Hunt:** Tool to sniff, hijack, and reset network connections.
7. **T-Sight:** A Tool for easily hijacking sessions and monitoring network connections.

8. **Zaproxy:** Web application security scanner and intercepting proxy.
9. **Paros:** Web proxy and vulnerability scanner for security testing.
10. **Shijack:** TCP/IP hijacking tools for network session takeover.
11. **Juggernaut:** Tool with capabilities for session hijacking and network manipulation.
12. **Hamster & Ferret:** Tools designed for HTTP session hijacking and sniffing.

A CSRF attack can force the user to perform state-changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

CSRF Scenario:

1. Visit your bank's site, log in.
2. Then visit the attacker's site (e.g. sponsored ad from an untrusted organization).
3. Attacker's page includes form with same fields as the bank's "Transfer Funds" form.
4. Form fields are pre-filled to transfer money from your account to attacker's account.
5. Attacker's page includes Javascript that submits form to your bank.
6. When the form gets submitted, the browser includes your cookies for the bank site, including the session token.
7. Bank transfers money to the attacker's account.
8. The form can be in an invisible iframe, so you never know the attack occurred.

Session Fixation

Session Fixation is an attack that permits an attacker to hijack a valid user session. The attack explores a limitation in the way the web application manages the session ID, more specifically, the vulnerable web application.

Session fixation Scenario:

1. The attacker accesses the web application login page and receives a session ID generated by the web application.
2. The attacker uses an additional technique, such as CRLF Injection, a man-in-the-middle attack, social engineering, etc., and gets the victim to use the provided session identifier.
3. The victim accesses the web application login page and logs in to the application. After authenticating, the web application treats anyone who uses this session ID as if they were this user.
4. The attacker uses the session ID to access the web application, take over the user session, and impersonate the victim.

Session vs Spoofing Attacks

Spoofing

- Method: Attacker steals user credentials (username/password)
- Process: Initiates a completely new session using stolen credentials
- Detection: May trigger account lockout or suspicious login alerts
- Persistence: Requires maintaining access to credentials

Hijacking

- Method: Attacker steals active session IDs
- Process: Uses existing authenticated session without re-authentication
- Detection: Harder to detect as uses legitimate session
- Persistence: Only valid for session duration

Session-Related Attack Techniques

Insecure Direct Object Reference (IDOR)

Attackers modify session IDs to gain access to other active sessions by analysing session ID formats and patterns.

Example Attack Flow:

1. Analyse session ID structure
2. Identify predictable patterns
3. Modify session ID to target other users
4. Access unauthorised resources

Session Fixation Attack

Exploits applications that assign session IDs before user authentication and fail to regenerate them after successful login.

Vulnerability Conditions:

- Session ID assigned before credential validation
- Session ID not modified after successful authentication
- Application accepts externally provided session IDs

Attack Process:

1. Attacker opens target website and obtains session ID (e.g., 12345678)
2. Attacker sends URL with this session ID to victim
3. Victim opens URL and authenticates with valid credentials
4. Server considers requests with that session ID as legitimate user requests

5. Attacker uses the same session ID to access victim's account

Tools and Techniques

Burp Suite

Comprehensive web application security testing platform with advanced session analysis capabilities.

Session Hijacking Features:

1. Session Token Analysis

- Intercept requests in Proxy tab
- Send session tokens to Sequencer for randomness testing
- Use Repeater to test session manipulation
- Analyse session management with Spider results

2. Session Fixation Testing

- Capture pre-authentication session ID
- Authenticate with valid credentials
- Compare pre-/post-authentication session IDs
- Test session acceptance from external sources

OWASP ZAP

Open-source web application security scanner with session management testing capabilities.

Usage Examples:

Automated session testing

- `zap.sh -daemon -port 8080`
- `zap-cli quick-scan --spider http://target-application.com`
- `zap-cli active-scan http://target-application.com`

Session analysis

→ Configure ZAP as proxy (localhost:8080)

Navigate through the application to capture session flows

Review session management findings in the report

Cookie Cadger

A passive session hijacking tool that captures and replays session cookies from wireless networks.

Usage Process:

Passive session capture

→java -jar CookieCadger.jar

Configuration steps:

1. Select wireless interface for monitoring
2. Start packet capture for HTTP traffic
3. Filter for session-related cookies
4. Save captured sessions for replay testing
5. Test session validity and access levels

AUTHORISATION V/S AUTHENTICATION

Authentication and authorisation are two different steps in access control:

- Authentication = *Who are you?*
Verifies identity (e.g., username/password, OTP, biometrics). It checks that the user is really who they claim to be.
- Authorisation = *What are you allowed to do?*
Decides what resources/actions an authenticated user can access (e.g., which pages, APIs, files, privileges).
- Order: Authentication comes first, then authorisation.
- Shortcut memory hook:
 - Authentication → identity
 - Authorisation → permissions/privileges

Cybersecurity Terms and Definitions

Authentication

Process of verifying the identity of a user or system, typically through credentials like username/password combinations.

Authorization

Process of determining what resources and actions an authenticated user is permitted to access.

Cookie Attributes

Security settings for HTTP cookies including HttpOnly, Secure, SameSite, and expiration parameters.

Cross-Site Request Forgery (CSRF)

Attack that tricks authenticated users into performing unintended actions on web applications.

Cross-Site Scripting (XSS)

Vulnerability allowing attackers to inject malicious scripts into web applications viewed by other users.

Entropy

Measure of randomness in data; higher entropy indicates better unpredictability and security.

JSON Web Token (JWT)

Open standard for securely transmitting information between parties as compact, URL-safe tokens.

Man-in-the-Middle (MITM)

Attack where attackers secretly intercept and potentially alter communications between two parties.

Session Fixation

Attack where an attacker fixes a user's session ID before authentication and then hijacks the session after login.

Session Prediction

Technique of analyzing session ID patterns to predict valid session tokens for unauthorized access.

Session Token

Unique identifier used to maintain user authentication state across multiple HTTP requests.

Session Timeout

Security mechanism that automatically terminates user sessions after a period of inactivity.

Stateless Authentication

Authentication method that doesn't require server-side session storage, typically using tokens like JWTs.

Token Rotation

Security practice of regularly generating new session tokens to limit the exposure window.

Traffic Analysis

Technique of examining network communication patterns to extract sensitive information like session tokens.

Countermeasures

Session IDS

- ☐ Using unpredictable (randomised) Session IDs
- ☐ Never use URLs with Session IDs
- ☐ Don't Re-use Session IDs
- Use HTTP-Only on Cookies, preventing XSS (Cross-Site Scripting)
- Don't use HTTP protocol without encryption --> Use TLS/SSL [HTTPS]
- Limiting incoming connections
- Minimising remote access
- Regenerating the session key after authentication
- Time - absolute / inactive *(e.g: 1h of inactivity the user will automatically log off)*
- Use MFA
- Use IPSec to encrypt

IPSec

Transport Mode - payload and ESP trailer are encrypted; IP header is not

Tunnel mode - everything is encrypted; cannot be used with NAT

Architecture Protocols

Authentication Header - guarantees the integrity and authentication of IP packet sender

Encapsulating Security Payload(ESP) - provides origin authenticity and integrity as well as confidentiality

Internet Key Exchange(IKE) - produces the keys for the encryption process

Oakley- uses Diffie-Hellman to create master and session keys

Internet Security Association Key Management Protocol (ISAKMP) - software that facilitates encrypted communication between two endpoints